

GitHub Actions Complete Lab Document

Goal

Build a complete CI/CD lab using GitHub Actions that covers almost everything normally done in Jenkins:

```
Developer Push
↓
GitHub Repository
↓
GitHub Actions Workflow
↓
Checkout
↓
Build
↓
Test
↓
SAST / Code Quality
↓
Docker Build
↓
Container Scan
↓
Artifact Upload
↓
Push Image to Registry
↓
Deploy to Kubernetes
↓
Rollout Verification
↓
Approval for Production
↓
Notification
```

1. GitHub Actions vs Jenkins Lab Mapping

Jenkins Concept	GitHub Actions Equivalent
Jenkinsfile	<code>.github/workflows/ci-cd.yml</code>
Jenkins Job	GitHub Workflow

Jenkins Concept	GitHub Actions Equivalent
Jenkins Stage	GitHub Job / Step
Jenkins Agent	GitHub Runner
Jenkins Credentials	GitHub Secrets
Jenkins Parameters	<code>workflow_dispatch</code> inputs
Jenkins Artifacts	<code>actions/upload-artifact</code>
Jenkins Post Actions	<code>if: success()</code> / <code>if: failure()</code> / job status
Jenkins Approval Stage	GitHub Environment Approval
Jenkins Plugins	GitHub Marketplace Actions
Jenkins Build Number	<code>github.run_number</code>
Jenkins Workspace Cleanup	Runner workspace auto-clean / manual cleanup
Jenkins Webhook	GitHub native event trigger

2. Required Lab Tools

Local / System Tools

Tool	Purpose
Git	Push code to GitHub
Docker Desktop	Build and test Docker images locally
Kubernetes / kind	Local Kubernetes cluster
kubectl	Deploy and verify Kubernetes resources
VS Code	Code editing

GitHub Side

Item	Purpose
GitHub Repository	Store source code
GitHub Actions	Run CI/CD pipeline
GitHub Secrets	Store passwords/tokens
GitHub Environments	Approval and deployment protection
GitHub Artifacts	Store build output/logs

Optional Enterprise Tools

Tool	Purpose
Docker Hub	Container image registry
GitHub Packages	GitHub-native registry
SonarQube	SAST/code quality
Trivy	Container/image/IaC scan
Checkov	Terraform/IaC security scan
Infracost	Cloud cost estimation
OpenCost	Kubernetes cost visibility
Slack/Teams/Email	Notifications

3. Repository Structure

Create this structure:

```
enterprise-cloud-platform/  
├── app/  
│   ├── index.html  
│   └── package.json  
├── Dockerfile  
├── k8s/  
│   ├── deployment.yaml  
│   └── service.yaml  
├── scripts/  
│   └── test.sh  
└── .github/  
    ├── workflows/  
    └── ci-cd.yml
```

4. Sample Application

app/index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>GitHub Actions CI/CD Lab</title>
</head>
<body>
  <h1>GitHub Actions CI/CD Deployment Successful</h1>
  <p>Version: v1</p>
</body>
</html>
```

app/package.json

```
{
  "name": "github-actions-lab",
  "version": "1.0.0",
  "scripts": {
    "test": "echo \"Basic test passed\" && exit 0"
  }
}
```

5. Dockerfile

```
FROM nginx:alpine
COPY app/index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

Local Test

```
docker build -t github-actions-lab:v1 .
docker run -d -p 8088:80 --name github-actions-lab github-actions-lab:v1
curl http://localhost:8088
```

Expected output:

```
GitHub Actions CI/CD Deployment Successful
```

Cleanup:

```
docker rm -f github-actions-lab
```

6. Kubernetes Manifests

k8s/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: github-actions-lab
spec:
  replicas: 2
  selector:
    matchLabels:
      app: github-actions-lab
  template:
    metadata:
      labels:
        app: github-actions-lab
    spec:
      containers:
        - name: github-actions-lab
          image: DOCKER_IMAGE_PLACEHOLDER
          ports:
            - containerPort: 80
```

k8s/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: github-actions-lab-service
spec:
  type: NodePort
  selector:
    app: github-actions-lab
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
```

7. GitHub Secrets Required

Go to:

GitHub Repository → Settings → Secrets and variables → Actions → New repository secret

Create these secrets:

Secret Name	Purpose
<code>DOCKERHUB_USERNAME</code>	Docker Hub username
<code>DOCKERHUB_TOKEN</code>	Docker Hub access token
<code>KUBE_CONFIG</code>	Kubernetes kubeconfig content
<code>SONAR_TOKEN</code>	Optional SonarQube token
<code>SLACK_WEBHOOK_URL</code>	Optional Slack notification

For local kind Kubernetes, get kubeconfig:

```
kubectl config view --raw
```

Copy full output and save it as GitHub secret:

```
KUBE_CONFIG
```

Important: Local kind cluster must be reachable from the GitHub runner. For local lab, Kubernetes deployment works best with a self-hosted GitHub runner installed on your own machine or Jenkins server machine.

8. Basic GitHub Actions Pipeline

Create file:

```
.github/workflows/ci-cd.yml
```

```
name: GitHub Actions CI/CD Lab

on:
  push:
    branches:
```

```
    - main
workflow_dispatch:

jobs:
  basic-check:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Show Repository Files
        run: ls -la

      - name: Basic Pipeline Test
        run: echo "GitHub Actions pipeline is working"
```

Test Level 1

Push code:

```
git add .
git commit -m "add basic github actions workflow"
git push origin main
```

Verify:

GitHub Repository → Actions → GitHub Actions CI/CD Lab → Latest run

Expected result:

```
basic-check passed
```

9. Build and Test Stage

Update workflow:

```
name: GitHub Actions CI/CD Lab

on:
  push:
    branches:
      - main
```

```

workflow_dispatch:

jobs:
  build-test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Install Dependencies
        working-directory: app
        run: npm install

      - name: Run Tests
        working-directory: app
        run: npm test

```

Test Level 2

Push and check GitHub Actions.

Expected:

```

Install Dependencies passed
Run Tests passed

```

Jenkins equivalent:

```

stage('Build')
stage('Test')

```

10. Artifact Upload

Artifacts are used to save build output, logs, reports, test results, HTML packages, or scan reports.

Add this job step:


```

- name: Prepare Artifact
  run: |
    mkdir -p artifact
    cp app/index.html artifact/index.html
    echo "Build Number: ${github.run_number}" > artifact/build-
info.txt
    echo "Commit SHA: ${github.sha}" >> artifact/build-info.txt

- name: Upload Build Artifact
  uses: actions/upload-artifact@v4
  with:
    name: webapp-build-${github.run_number}
    path: artifact/

```

Test Level 3

After workflow run:

GitHub Actions Run → Summary → Artifacts

Expected:

webapp-build-<run_number>

Jenkins equivalent:

archiveArtifacts artifacts: 'artifact/**'

11. Docker Build Stage

```

docker-build:
  runs-on: ubuntu-latest
  needs: build-test

  steps:
    - name: Checkout Code
      uses: actions/checkout@v4

    - name: Build Docker Image
      run: |
        docker build -t github-actions-lab:${github.run_number} .
        docker images

```

Test Level 4

Expected:

```
github-actions-lab:<run_number>
```

Jenkins equivalent:

```
stage('Docker Build')
```

12. Avoid Same v1 Tag Issue

Do not always use:

```
v1  
latest
```

Better tags:

```
${{ github.run_number }}  
${{ github.sha }}
```

Recommended image tag:

```
username/appname:${{ github.run_number }}
```

or:

```
username/appname:${{ github.sha }}
```

Example:

```
IMAGE_TAG: ${{ github.run_number }}
```

This solves the same issue you saw in Jenkins where the same `v1` tag causes confusion during deployment.

13. Docker Hub Push

```
docker-push:
  runs-on: ubuntu-latest
  needs: build-test

  env:
    IMAGE_NAME: ${ secrets.DOCKERHUB_USERNAME }/github-actions-lab
    IMAGE_TAG: ${ github.run_number }

  steps:
    - name: Checkout Code
      uses: actions/checkout@v4

    - name: Login to Docker Hub
      uses: docker/login-action@v3
      with:
        username: ${ secrets.DOCKERHUB_USERNAME }
        password: ${ secrets.DOCKERHUB_TOKEN }

    - name: Build Docker Image
      run: docker build -t $IMAGE_NAME:$IMAGE_TAG .

    - name: Push Docker Image
      run: docker push $IMAGE_NAME:$IMAGE_TAG
```

Test Level 5

Check Docker Hub repository.

Expected:

```
username/github-actions-lab:<run_number>
```

Jenkins equivalent:

```
withCredentials(...)
docker login
docker build
docker push
```

14. Trivy Container Scan

```
- name: Run Trivy Image Scan
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: ${ env.IMAGE_NAME }:${ env.IMAGE_TAG }
    format: table
    exit-code: '0'
```

For strict security gate:

```
exit-code: '1'
severity: 'CRITICAL,HIGH'
```

Test Level 6

Expected:

```
Trivy scan completed
```

Jenkins equivalent:

```
stage('Container Scan')
```

15. SAST / Code Quality Scan

Option A: GitHub CodeQL

```
codeql-analysis:
  runs-on: ubuntu-latest
  permissions:
    security-events: write
    contents: read

  steps:
    - name: Checkout Code
      uses: actions/checkout@v4

    - name: Initialize CodeQL
      uses: github/codeql-action/init@v3
      with:
```

```
languages: javascript

- name: Perform CodeQL Analysis
  uses: github/codeql-action/analyze@v3
```

Option B: SonarQube

```
- name: SonarQube Scan
  uses: SonarSource/sonarqube-scan-action@master
  env:
    SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
    SONAR_HOST_URL: ${ secrets.SONAR_HOST_URL }
```

Test Level 7

Check:

```
GitHub → Security tab
or
SonarQube dashboard
```

Jenkins equivalent:

```
stage('SAST')
```

16. IaC Security Scan with Checkov

If Terraform files exist:

```
iac-scan:
  runs-on: ubuntu-latest

  steps:
    - name: Checkout Code
      uses: actions/checkout@v4

    - name: Run Checkov
      uses: bridgecrewio/checkov-action@master
      with:
        directory: .
        soft_fail: true
```

For strict gate:

```
soft_fail: false
```

Test Level 8

Expected:

```
Checkov scan completed
```

Jenkins equivalent:

```
stage('Terraform Security Scan')
```

17. Policy / Governance Stage

Policy tools:

Tool	Use Case
OPA	Policy-as-code
Conftest	Test Kubernetes/Terraform YAML policies
Sentinel	Terraform Cloud/Enterprise policy
GitHub Branch Protection	Prevent direct production changes
GitHub Environment Rules	Approval before deployment

Example governance rule:

```
Production deployment requires approval from HOD / Software Head.
```

GitHub way:

```
Repository → Settings → Environments → New environment → production
```

Enable:

```
Required reviewers
```

Add:

18. Manual Parameters Like Jenkins

Jenkins has parameters. GitHub Actions has `workflow_dispatch` inputs.

```
on:
  workflow_dispatch:
    inputs:
      environment:
        description: 'Choose deployment environment'
        required: true
        default: 'dev'
        type: choice
        options:
          - dev
          - uat
          - prod
      namespace:
        description: 'Kubernetes namespace'
        required: true
        default: 'default'
      image_tag:
        description: 'Image tag to deploy'
        required: false
```

Test Level 9

Go to:

GitHub → Actions → Workflow → Run workflow

Expected:

You can choose dev / uat / prod

Jenkins equivalent:

```
parameters {
  choice(name: 'ENV', choices: ['dev', 'uat', 'prod'])
}
```

19. Kubernetes Deployment Stage

For GitHub-hosted runner, Kubernetes cluster must be publicly reachable. For local kind cluster, use self-hosted GitHub runner.

```
deploy-kubernetes:
  runs-on: self-hosted
  needs: docker-push
  environment: dev

  env:
    IMAGE_NAME: ${ secrets.DOCKERHUB_USERNAME }}/github-actions-lab
    IMAGE_TAG: ${ github.run_number }

  steps:
    - name: Checkout Code
      uses: actions/checkout@v4

    - name: Setup Kubeconfig
      run: |
        mkdir -p ~/.kube
        echo "${ secrets.KUBE_CONFIG }}" > ~/.kube/config

    - name: Update Kubernetes Manifest Image
      run: |
        sed -i "s|DOCKER_IMAGE_PLACEHOLDER|$IMAGE_NAME:$IMAGE_TAG|g" k8s/
deployment.yaml
        cat k8s/deployment.yaml

    - name: Deploy to Kubernetes
      run: |
        kubectl apply -f k8s/deployment.yaml
        kubectl apply -f k8s/service.yaml

    - name: Rollout Status
      run: kubectl rolloutstatus deployment/github-actions-lab

    - name: Verify Pods
      run: kubectl get pods -o wide

    - name: Verify Service
      run: kubectl get svc
```

Test Level 10

On your local system:


```
kubectl get pods
kubectl get svc
kubectl rollout status deployment/github-actions-lab
```

Expected:

```
github-actions-lab pods running
NodePort service available on 30080
```

Browser test:

```
http://localhost:30080
```

Jenkins equivalent:

```
stage('Kubernetes Deploy')
stage('Rollout Status')
```

20. Production Approval Stage

```
deploy-production:
  runs-on: self-hosted
  needs: docker-push
  environment: production

  steps:
    - name: Production Deployment
      run: echo "Deploying to production after approval"
```

GitHub approval setup:

```
Repository → Settings → Environments → production → Required reviewers
```

Add reviewers:

```
HOD
Software Head
DevOps Manager
```

Test Level 11

Run production workflow.

Expected:

```
Workflow waits for approval before production deployment
```

Jenkins equivalent:

```
input message: 'Approve production deployment?'
```

21. Notification Stage

Slack Notification Example

```
- name: Slack Notification
  if: always()
  uses: 8398a7/action-slack@v3
  with:
    status: ${ job.status }
    fields: repo,message,commit,author,action,eventName,ref,workflow
  env:
    SLACK_WEBHOOK_URL: ${ secrets.SLACK_WEBHOOK_URL }
```

Simple Email Alternative

GitHub does not provide native email sending like Jenkins email plugin. Options:

Option	Use
Slack	Team alert
Teams Webhook	Enterprise alert
SMTP Action	Email alert
GitHub notification	Basic built-in alert

Jenkins equivalent:

```
post {
  success { mail ... }
```

```
    failure { mail ... }  
  }
```

22. Full Enterprise GitHub Actions Pipeline

```
name: Enterprise GitHub Actions CI/CD Lab  
  
on:  
  push:  
    branches:  
      - main  
  workflow_dispatch:  
    inputs:  
      environment:  
        description: 'Choose environment'  
        required: true  
        default: 'dev'  
        type: choice  
        options:  
          - dev  
          - uat  
          - prod  
      namespace:  
        description: 'Kubernetes namespace'  
        required: true  
        default: 'default'  
  
jobs:  
  build-test:  
    runs-on: ubuntu-latest  
  
    steps:  
      - name: Checkout Code  
        uses: actions/checkout@v4  
  
      - name: Setup Node.js  
        uses: actions/setup-node@v4  
        with:  
          node-version: '20'  
  
      - name: Install Dependencies  
        working-directory: app  
        run: npm install  
  
      - name: Run Tests  
        working-directory: app  
        run: npm test
```

```

- name: Prepare Artifact
  run: |
    mkdir -p artifact
    cp app/index.html artifact/index.html
    echo "Build Number: ${github.run_number}" > artifact/build-
info.txt
    echo "Commit SHA: ${github.sha}" >> artifact/build-info.txt

- name: Upload Build Artifact
  uses: actions/upload-artifact@v4
  with:
    name: webapp-build-${github.run_number}
    path: artifact/

codeql-analysis:
  runs-on: ubuntu-latest
  permissions:
    security-events: write
    contents: read

  steps:
    - name: Checkout Code
      uses: actions/checkout@v4

    - name: Initialize CodeQL
      uses: github/codeql-action/init@v3
      with:
        languages: javascript

    - name: Perform CodeQL Analysis
      uses: github/codeql-action/analyze@v3

docker-build-scan-push:
  runs-on: ubuntu-latest
  needs: build-test

  env:
    IMAGE_NAME: ${secrets.DOCKERHUB_USERNAME}/github-actions-lab
    IMAGE_TAG: ${github.run_number}

  steps:
    - name: Checkout Code
      uses: actions/checkout@v4

    - name: Login to Docker Hub
      uses: docker/login-action@v3
      with:
        username: ${secrets.DOCKERHUB_USERNAME}
        password: ${secrets.DOCKERHUB_TOKEN}

    - name: Build Docker Image

```

```

    run: docker build -t $IMAGE_NAME:$IMAGE_TAG .

- name: Trivy Image Scan
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: ${ env.IMAGE_NAME }:${ env.IMAGE_TAG }
    format: table
    exit-code: '0'

- name: Push Docker Image
  run: docker push $IMAGE_NAME:$IMAGE_TAG

deploy-dev:
  runs-on: self-hosted
  needs: docker-build-scan-push
  if: ${ github.event.inputs.environment == 'dev' || github.event_name ==
'push' }}
  environment: dev

  env:
    IMAGE_NAME: ${ secrets.DOCKERHUB_USERNAME }/github-actions-lab
    IMAGE_TAG: ${ github.run_number }
    NAMESPACE: ${ github.event.inputs.namespace || 'default' }

  steps:
    - name: Checkout Code
      uses: actions/checkout@v4

    - name: Setup Kubeconfig
      run: |
        mkdir -p ~/.kube
        echo "${ secrets.KUBE_CONFIG }}" > ~/.kube/config

    - name: Update Image in Manifest
      run: |
        sed -i "s|DOCKER_IMAGE_PLACEHOLDER|$IMAGE_NAME:$IMAGE_TAG|g" k8s/
deployment.yaml

    - name: Deploy to Kubernetes
      run: |
        kubectl apply -n $NAMESPACE -f k8s/deployment.yaml
        kubectl apply -n $NAMESPACE -f k8s/service.yaml

    - name: Rollout Status
      run: kubectl rollout status -n $NAMESPACE deployment/github-actions-
lab

    - name: Verify Deployment
      run: |
        kubectl get pods -n $NAMESPACE -o wide
        kubectl get svc -n $NAMESPACE

```

```

deploy-prod:
  runs-on: self-hosted
  needs: docker-build-scan-push
  if: ${ github.event.inputs.environment == 'prod' }}
  environment: production

  env:
    IMAGE_NAME: ${ secrets.DOCKERHUB_USERNAME }}/github-actions-lab
    IMAGE_TAG: ${ github.run_number }}
    NAMESPACE: ${ github.event.inputs.namespace || 'default' }}

  steps:
    - name: Checkout Code
      uses: actions/checkout@v4

    - name: Setup Kubeconfig
      run: |
        mkdir -p ~/.kube
        echo "${ secrets.KUBE_CONFIG }}" > ~/.kube/config

    - name: Update Image in Manifest
      run: |
        sed -i "s|DOCKER_IMAGE_PLACEHOLDER|$IMAGE_NAME:$IMAGE_TAG|g" k8s/
deployment.yaml

    - name: Deploy to Production Kubernetes
      run: |
        kubectl apply -n $NAMESPACE -f k8s/deployment.yaml
        kubectl apply -n $NAMESPACE -f k8s/service.yaml

    - name: Production Rollout Status
      run: kubectl rollout status -n $NAMESPACE deployment/github-actions-
lab

```

23. Testing Checklist at Every Level

Level	Test	Command / Location	Expected Result
1	Workflow trigger	GitHub → Actions	Workflow starts
2	Checkout	Workflow logs	Code downloaded
3	Build/Test	<code>npm test</code>	Test passed
4	Artifact	Run summary	Artifact uploaded
5	Docker build	Workflow logs	Image built

Level	Test	Command / Location	Expected Result
6	Trivy scan	Workflow logs	Scan completed
7	Docker push	Docker Hub	New image tag visible
8	Kubernetes deploy	<code>kubectl get deploy</code>	Deployment created
9	Pod health	<code>kubectl get pods</code>	Pods Running
10	Rollout	<code>kubectl rollout status deployment/github-actions-lab</code>	Successfully rolled out
11	Service	<code>kubectl get svc</code>	NodePort 30080 visible
12	Browser	<code>http://localhost:30080</code>	App opens
13	Approval	GitHub Environment	Waiting for reviewer
14	Failure test	Break test command	Pipeline fails
15	Rollback test	Deploy old image tag	Previous version restored

24. Failure Testing

Test Failed Build

Change test script:

```
"test": "echo \"Test failed\" && exit 1"
```

Expected:

```
Pipeline failed at test stage
Docker build should not run
Deployment should not happen
```

Test Failed Docker Build

Break Dockerfile:

```
FROM wrong-image-name
```

Expected:

```
Docker build failed  
Image not pushed  
Deployment not started
```

Test Failed Kubernetes Deploy

Use wrong image name:

```
image: wrongimage:test
```

Expected:

```
Pods may go into ImagePullBackOff  
Rollout status fails
```

Verify:

```
kubectl get pods  
kubectl describe pod <pod-name>
```

25. Rollback Method

Check rollout history

```
kubectl rollout history deployment/github-actions-lab
```

Rollback to previous version

```
kubectl rollout undo deployment/github-actions-lab
```

Check rollout status

```
kubectl rollout status deployment/github-actions-lab
```

Jenkins equivalent:

```
stage('Rollback')
```


GitHub Actions can also create a manual rollback workflow later.

26. Best Practices

Image Tagging

Use:

```
{{ github.run_number }}  
{{ github.sha }}
```

Avoid:

```
latest  
v1
```

Secrets

Never hardcode:

```
Docker password  
Kubeconfig  
Cloud credentials  
Tokens
```

Use:

```
GitHub Secrets
```

Approval

Use:

```
GitHub Environments
```

for:

```
prod approval  
manual gate  
reviewer control
```

Security

Minimum security checks:

```
CodeQL / SonarQube
Trivy
Checkov
Secret scanning
Branch protection
```

Governance

Use:

```
Branch protection
Required PR review
Environment reviewers
CODEOWNERS
OPA / Conftest
```

FinOps

Use:

```
Infracost for Terraform cost estimation
OpenCost for Kubernetes cost monitoring
Cloud billing alerts
```

27. Final Interview Explanation

You can explain GitHub Actions like this:

```
GitHub Actions is a CI/CD automation platform built into GitHub. Since the
source code already lives in GitHub, it becomes easy to trigger pipelines
directly from push, pull request, tag, or manual workflow events.
```

```
It can perform the same major activities as Jenkins, such as checkout, build,
test, artifact upload, Docker build, security scanning, image push,
Kubernetes deployment, approval gates, and notifications.
```

```
The main difference is that GitHub Actions is more cloud-native and
repository-centric, while Jenkins is more centralized and highly
```

customizable. In modern DevOps, GitHub Actions is commonly used with Docker, Kubernetes, Terraform, Trivy, SonarQube, Checkov, and ArgoCD.

28. Final Practical Roadmap

Follow this order:

```
Step 1: Basic workflow
Step 2: Build and test
Step 3: Artifact upload
Step 4: Docker build
Step 5: Docker Hub push
Step 6: Trivy scan
Step 7: Kubernetes deploy
Step 8: Rollout status
Step 9: Approval stage
Step 10: Notification
Step 11: Rollback workflow
Step 12: Add Terraform + Checkov + Infracost
```

29. Your Current Jenkins Pending Tasks Converted to GitHub Actions

Jenkins Pending Task	GitHub Actions Implementation
Fix dynamic image tag	Use <code>\${{ github.run_number }}</code> or <code>\${{ github.sha }}</code>
Avoid same v1 tag	Never use static tag for deployment
Checkout	<code>actions/checkout@v4</code>
Build	Normal shell commands or language setup actions
Test	<code>npm test</code> , <code>pytest</code> , <code>mvn test</code> , etc.
Docker Build	<code>docker build</code>
Kubernetes Deploy	<code>kubectl apply</code>
Rollout Status	<code>kubectl rollout status</code>
Success/Failure post actions	<code>if: success()</code> / <code>if: failure()</code> / <code>if: always()</code>
Cleanup workspace	Auto-cleaned runner or manual <code>rm -rf</code>
Docker Hub credentials	GitHub Secrets

Jenkins Pending Task	GitHub Actions Implementation
GitHub token	Built-in <code>GITHUB_TOKEN</code>
ACR/ECR/GCR later	Cloud login actions
Archive artifact	<code>actions/upload-artifact@v4</code>
Trivy image scan	<code>aquasecurity/trivy-action</code>
Dependency/code scan	CodeQL / SonarQube / Snyk
Email/Slack/Teams alert	Webhook/action integration
Parameters	<code>workflow_dispatch</code> inputs
Dev/UAT/Prod deploy	GitHub Environments
Manual approval	Required reviewers
Prometheus/Grafana integration	Add monitoring verification stage

30. Final Target Architecture

```

GitHub Repository
  ↓
GitHub Actions
  ↓
Build + Test
  ↓
Artifact Upload
  ↓
SAST / Secret Scan / IaC Scan
  ↓
Docker Build
  ↓
Trivy Image Scan
  ↓
Docker Hub / ACR / ECR / GCR
  ↓
Kubernetes Deployment
  ↓
Rollout Status
  ↓
Approval Gate for Production
  ↓
Notification
  ↓
Monitoring / Cost / Governance

```